UNIT – 5

ASSESSMENT – 3

PYTHON CODE

NIHARIKA PREEJESH

192524506

1. # System Decomposition and Analysis

AI Medical Diagnosis System

```
def collect_input():
    symptoms = input("Enter symptoms separated by comma:
")
    return symptoms.lower().split(",")



def preprocess(symptoms):
    return [s.strip() for s in symptoms]



def feature_extraction(symptoms):
    features = {}

    features["fever"] = "fever" in symptoms
    features["cough"] = "cough" in symptoms
    features["breathlessness"] = "breathlessness" in symptoms
```

```
    features["rash"] = "rash" in symptoms

    return features


def diagnosis(features):

    if features["cough"] and features["breathlessness"]:
        return "Possible Pneumonia"

    elif features["fever"] and features["rash"]:
        return "Possible Measles"

    elif features["fever"] and features["cough"]:
        return "Possible Flu"

    else:
        return "No matching diagnosis"


def display_result(result):
    print("\nDiagnosis:", result)
```

```
# Main system

symptoms = collect_input()

symptoms = preprocess(symptoms)

features = feature_extraction(symptoms)

result = diagnosis(features)

display_result(result)
```

2. # Architecture Reconstruction

```
# Simple Recommendation System

def user_profile():
    name = input("Enter user name: ")
    interest = input("Enter interest: ")

    return name, interest.lower()
```

```
def content_database():
    return {
        "python": ["Python Programming", "Machine Learning", "Data Science"],
        "web": ["HTML", "CSS", "JavaScript"],
        "ai": ["Artificial Intelligence", "Machine Learning", "Deep Learning"]
    }


def recommendation_engine(interest, database):

    if interest in database:
        return database[interest]

    return ["No recommendations available"]


def display_recommendations(name, recommendations):
```

```
    print("\nUser:", name)
    print("Recommended Items:")

    for item in recommendations:
        print("-", item)


# Main system

name, interest = user_profile()

database = content_database()

recommendations = recommendation_engine(
    interest,
    database
)

display_recommendations(
    name,
    recommendations
)
```

3. # Algorithm Identification
Decision Tree-like Classification

```
def predict(attendance, study_hours):

    # Root decision
    if attendance >= 75:

        # Second decision
        if study_hours >= 3:
            return "Pass"
        else:
            return "Fail"

    else:
        return "Fail"



attendance = float(
    input("Enter attendance percentage: ")
)
```

```
study_hours = float(
    input("Enter study hours per day: ")
)

result = predict(attendance, study_hours)

print("\nPrediction:", result)
```

4. # Data Flow and Process Mapping

```
# Distributed Fraud Detection Simulation

def collect_transactions():

    transactions = [
        {
            "id": 101,
            "amount": 500,
            "location": "known"
        },
        {
```

```
            "id": 102,
            "amount": 50000,
            "location": "unknown"
        },
        {
            "id": 103,
            "amount": 2000,
            "location": "known"
        },
        {
            "id": 104,
            "amount": 75000,
            "location": "unknown"
        }
    ]

    return transactions


def preprocess(transaction):

    amount = transaction["amount"]
```

```
    location = transaction["location"]

    return amount, location


def detect_fraud(transaction):

    amount, location = preprocess(transaction)

    risk = 0

    if amount > 10000:
        risk += 1

    if location == "unknown":
        risk += 1

    if risk >= 2:
        return "High Risk"

    elif risk == 1:
        return "Medium Risk"
```

```
    else:
        return "Low Risk"


def process_data(transactions):

    results = []

    for transaction in transactions:

        status = detect_fraud(transaction)

        results.append(
            (transaction["id"], status)
        )

    return results


def display(results):
```

```
    print("\nFraud Detection Results")

    for transaction_id, status in results:

        print(
            "Transaction",
            transaction_id,
            ":",
            status
        )


# Main program

data = collect_transactions()

results = process_data(data)

display(results)
```

5. # Improvement and Redesign

Improved Recommendation System

```
cache = {}


def recommendation_engine(user, interest):

    key = user + "_" + interest

    # Check cache
    if key in cache:

        print("Result retrieved from cache.")

        return cache[key]

    # Generate recommendation
    if interest == "python":

        result = [
            "Python",
            "Machine Learning",
            "Data Science"
```

```
    ]

elif interest == "web":

    result = [
        "HTML",
        "CSS",
        "JavaScript"
    ]

elif interest == "ai":

    result = [
        "Artificial Intelligence",
        "Machine Learning",
        "Deep Learning"
    ]

else:

    result = [
        "General Programming"
```

```
        ]

    # Store result
    cache[key] = result

    return result


def display(result):

    print("\nRecommendations:")

    for item in result:
        print("-", item)


user = input("Enter user name: ")
interest = input("Enter interest: ").lower()

result = recommendation_engine(
    user,
    interest
```

```
)

display(result)

print("\nRequesting same recommendation again...")

result = recommendation_engine(
    user,
    interest
)

display(result)
```